Notas de Aula
Prof. Adriano Milanez

Programação Web I
Responsive Web Development
Front-End Design Engineering
Programação Web Responsiva

[aula 21]
[Atualizado em: 25/09/2025]

[VITE – Rotas Estáticas e Dinâmicas]

1

VITE – ROTAS

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS?**

  - ✓ **Rotas** são as instruções que determinam **qual conteúdo ou qual funcionalidade deve ser exibida ou executada** quando um usuário acessa uma determinada **URL** (endereço na barra do navegador).

  - ✓ **Como funcionam?**
    - ✓ Quando um usuário acessa uma URL como `https://meusite.com/contato`, o sistema (framework ou servidor) compara esse caminho com uma lista de rotas definidas.
    - ✓ Se encontrar uma correspondência, ele executa a ação associada àquela rota, como:
      - ✓ Renderizar uma página
      - ✓ Exibir um componente
      - ✓ Executar uma lógica (ex: buscar dados de um banco)

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS?**

  - ✓ **Classificação**

    1. **Estáticas**
       - ✓ Definidas com caminhos fixos.
       - ✓ Exemplo: `/home, /contato, /sobre`
    2. **Dinâmicas**
       - ✓ Contêm parâmetros variáveis.
       - ✓ Exemplo: `/produto/:id, /user/:username`
    3. **Públicas**
       - ✓ Acessíveis por qualquer usuário, mesmo sem login.
    4. **Privadas (Protegidas)**
       - ✓ Requerem autenticação/autorização para acesso.
       - ✓ Exemplo: `/dashboard, /perfil`
    5. **Aninhadas**
       - ✓ Rotas dentro de outras (subrotas).
       - ✓ Exemplo: `/admin/usuarios, /admin/`

1

VITE – ROTAS ESTÁTICAS

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- **O que são ROTAS ESTÁTICAS?**

  - São aquelas que têm endereços fixos e previsíveis, definidos diretamente no seu código. Elas não dependem de parâmetros ou dados dinâmicos.

  - São caminhos pré-definidos no código da sua aplicação, associados diretamente a um componente ou página específica.

  - Uma **rota estática** não muda com base em parâmetros, dados externos ou interações dinâmicas. Ela está "gravada" no código.

  - Para que consigamos navegar entre os componentes, precisamos criar **roteamento**.

  - **Roteamento** é a capacidade de mostrar diferentes páginas ao usuário. Isso significa que o usuário pode mover-se entre diferentes partes de um aplicativo inserindo um URL ou clicando em um elemento.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **ROTAS ESTÁTICAS**
  - ✓ **Características das Rotas Estáticas**
    - ✓ **Fixas:**
      - ✓ O caminho é sempre o mesmo, como /sobre, /contato, /servicos.
    - ✓ **Simples de configurar:**
      - ✓ Você define um caminho e associa um componente.
    - ✓ **Previsíveis:**
      - ✓ Ideais para conteúdo institucional ou páginas sem dados variáveis.
    - ✓ **Renderizam um componente específico:**
      - ✓ Cada URL leva sempre à mesma visualização.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **ROTAS ESTÁTICAS**
  - ✓ **Quando usar Rotas Estáticas**
    - ✓ **Use rotas estáticas quando**
      - ✓ **A página tem conteúdo fixo, como:**
        - ✓ Home
        - ✓ Sobre
        - ✓ Contato
        - ✓ Política de privacidade
        - ✓ Ajuda/FAQ
      - ✓ **Você não precisa de dados variáveis na URL**
      - ✓ **Quer organizar a navegação principal do seu app**

2

VITE – REACT-ROUTER-DOM

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

- ✓ Por padrão, o React vem sem **roteamento**, pois foi pensado no modelo de **Single Page Application** ou “Aplicação de Página Única", cuja funcionalidade está concentrada em uma única página. Porém, para tornar isso possível em nosso projeto, precisamos adicionar uma biblioteca chamada **react-router-dom**.

- ✓ O **react-router-dom** é uma **biblioteca** popular para gerenciar a navegação em aplicativos React. Ele fornece uma maneira declarativa e eficiente de definir rotas e vincular componentes a URLs específicos.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**

  - ✓ É a biblioteca de roteamento para aplicações React que rodam no navegador. Ela permite que sua aplicação se comporte como uma SPA (Single Page Application) — ou seja, troca de páginas sem recarregar o site.

  - ✓ **Principais responsabilidades**
    - ✓ Gerenciar o histórico de navegação
    - ✓ Sincronizar a URL do navegador com a interface exibida
    - ✓ Controlar qual componente deve ser exibido para cada rota

  - ✓ **Principais componentes**
    - ✓ BrowserRouter
    - ✓ Routes
    - ✓ Route
    - ✓ Link

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**

  - ✓ **BrowserRouter**
    - ✓ O ***BrowserRouter*** é o componente que ativa o roteamento na aplicação. Ele envolve a sua aplicação (ou parte dela) e fornece o contexto de roteamento para os demais componentes do React Router.

  - ✓ **Características**
    - ✓ Usa a History API do navegador (sem # na URL)
    - ✓ Permite que a aplicação controle o navegador via ***pushState***
    - ✓ Deve ser usado ***uma única vez***, geralmente no nível mais alto (ex: main.tsx ou App.tsx)

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **Routes**
    - ✓ O **Routes** funciona como um container para todas as suas definições de rotas.
    - ✓ **Características**
      - ✓ Seleciona automaticamente a primeira rota que casar com a URL atual.
      - ✓ Deve conter apenas <***Route***>***s*** como filhos

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**

  - ✓ **Route**
    - ✓ O **Route** é o elemento que define uma rota específica.
    - ✓ Cada <***Route***> mapeia um caminho da URL (***path***) para um componente (***element***) que será exibido quando aquele caminho for acessado.

  - ✓ **Características**
    - ✓ Tem uma propriedade ***path*** que define a URL da rota.
    - ✓ Usa a propriedade ***element*** para determinar o que será renderizado.
    - ✓ Pode ser estática (/sobre) ou dinâmica (/produtos/:id).
    - ✓ Pode ser aninhada para rotas internas (rotas filhas)

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**

  - ✓ **Link**
    - ✓ O componente **<Link>** é utilizado para criar navegação interna entre páginas em uma aplicação React que usa o **react-router-dom**.
    - ✓ Diferente do elemento HTML tradicional **<a>**, o **<Link>** não recarrega a página.
    - ✓ Em vez disso, ele altera a URL no navegador e atualiza o conteúdo exibido, mantendo o comportamento de uma SPA (Single Page Application).

    - ✓ **Características**
      - ✓ É usado apenas dentro de aplicações React com React Router.
      - ✓ Substitui o uso de **<a href="...">**.
      - ✓ Garante que a navegação ocorra de forma rápida e fluida, sem reload.
      - ✓ **Atualiza a URL sem sair da aplicação**.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ Esses quatro componentes trabalham juntos para fornecer um sistema de navegação completo dentro da aplicação React, sem que o navegador precise recarregar a página.

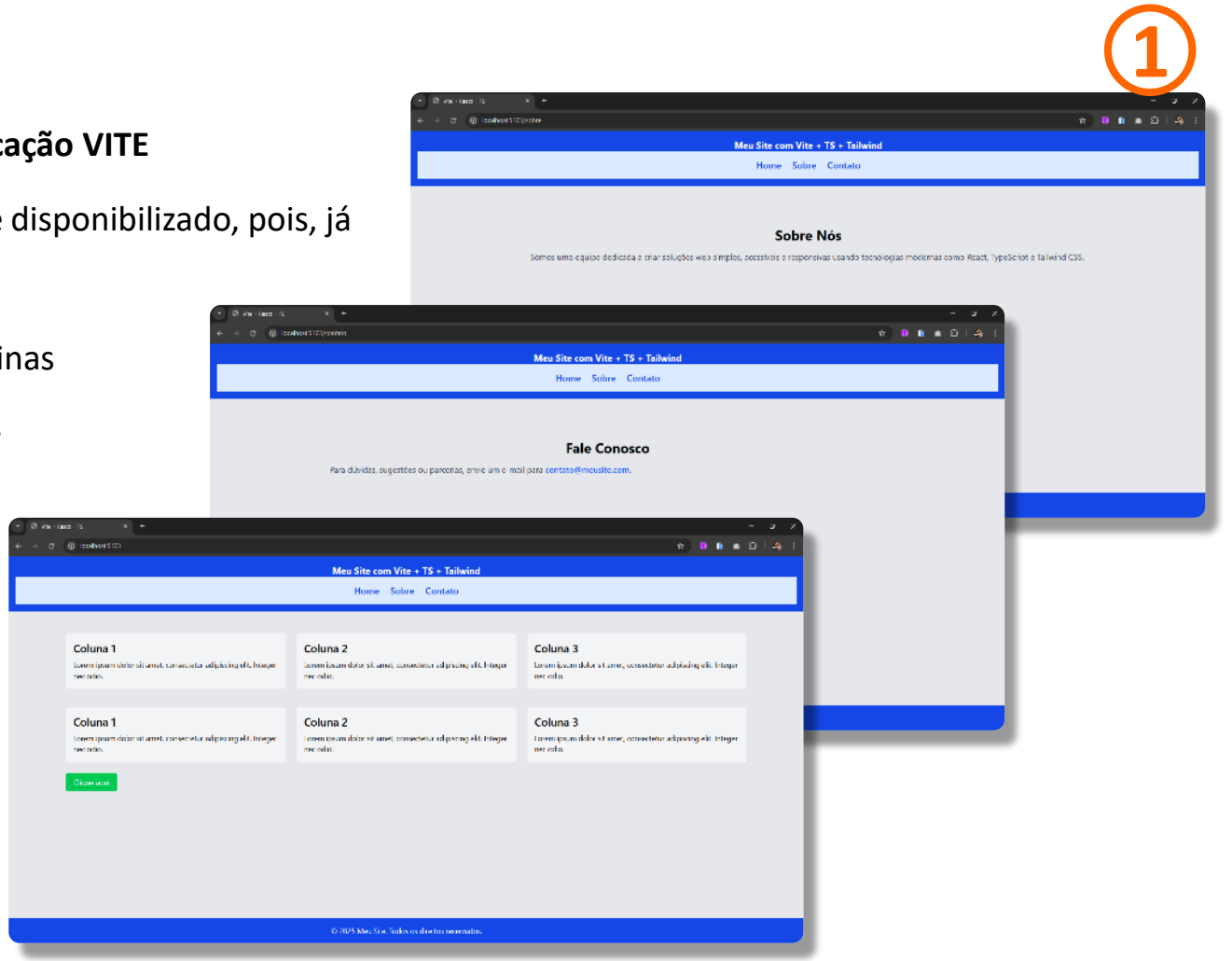
Resumo Teórico

Componente	Função
BrowserRouter	Ativa o roteamento usando a URL real do navegador
Routes	Agrupa e organiza as rotas; escolhe qual <Route> exibir
Route	Define qual componente exibir com base na URL (ex: /, /sobre)
Link	Cria links de navegação interna que alteram a URL **sem recarregar** a página

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

- ✓ **Vamos criar nossa OITAVA aplicação VITE**
  - ✓ Vamos utilizar o template disponibilizado, pois, já foi criado com Tailwind.
    1. Vamos criar as páginas como nas imagens.

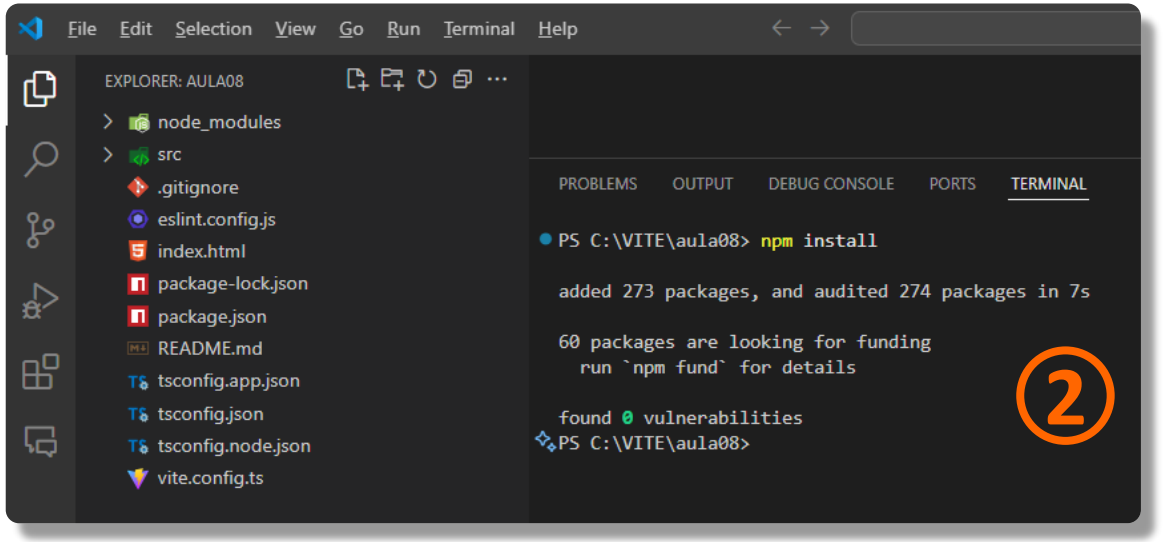


COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

- ✓ **Vamos criar nossa OITAVA aplicação VITE**
  - ✓ Vamos utilizar o template disponibilizado, pois, já foi criado com Tailwind.
    2. Vamos instalar as bibliotecas necessárias para nosso projeto, primeiro, instalamos o pacote NODE-MODULES

'npm install'



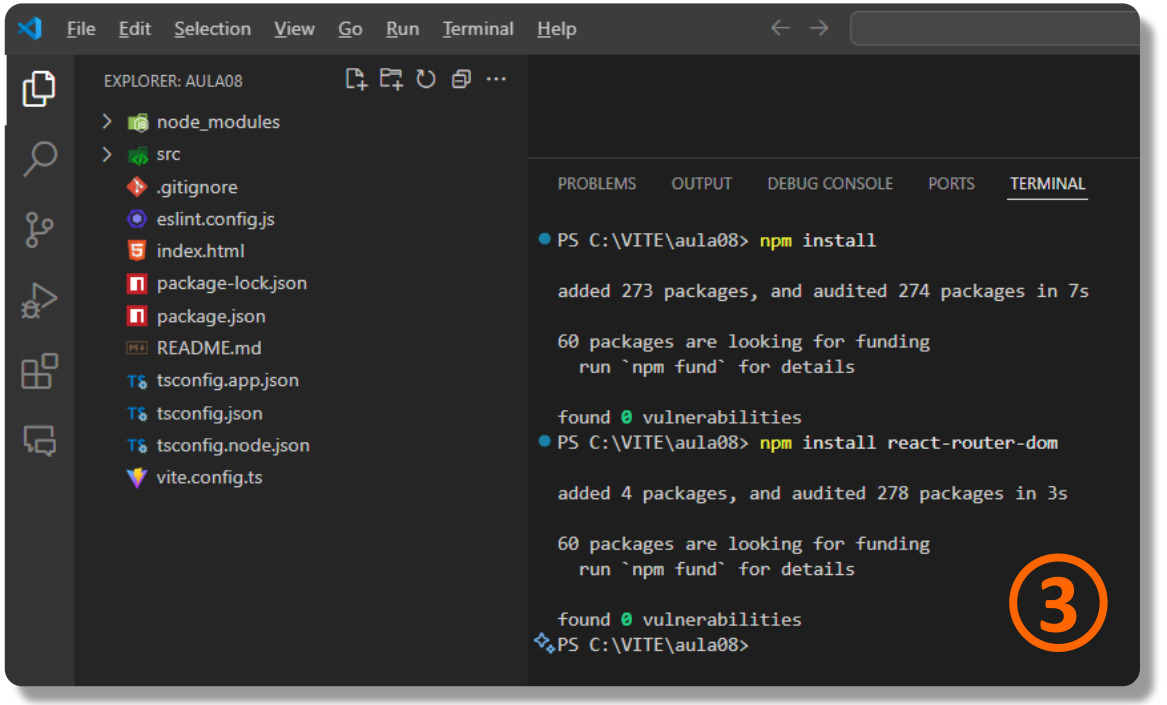
COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

3. Na sequência instalamos a biblioteca do **REACT-ROUTER-DOM**

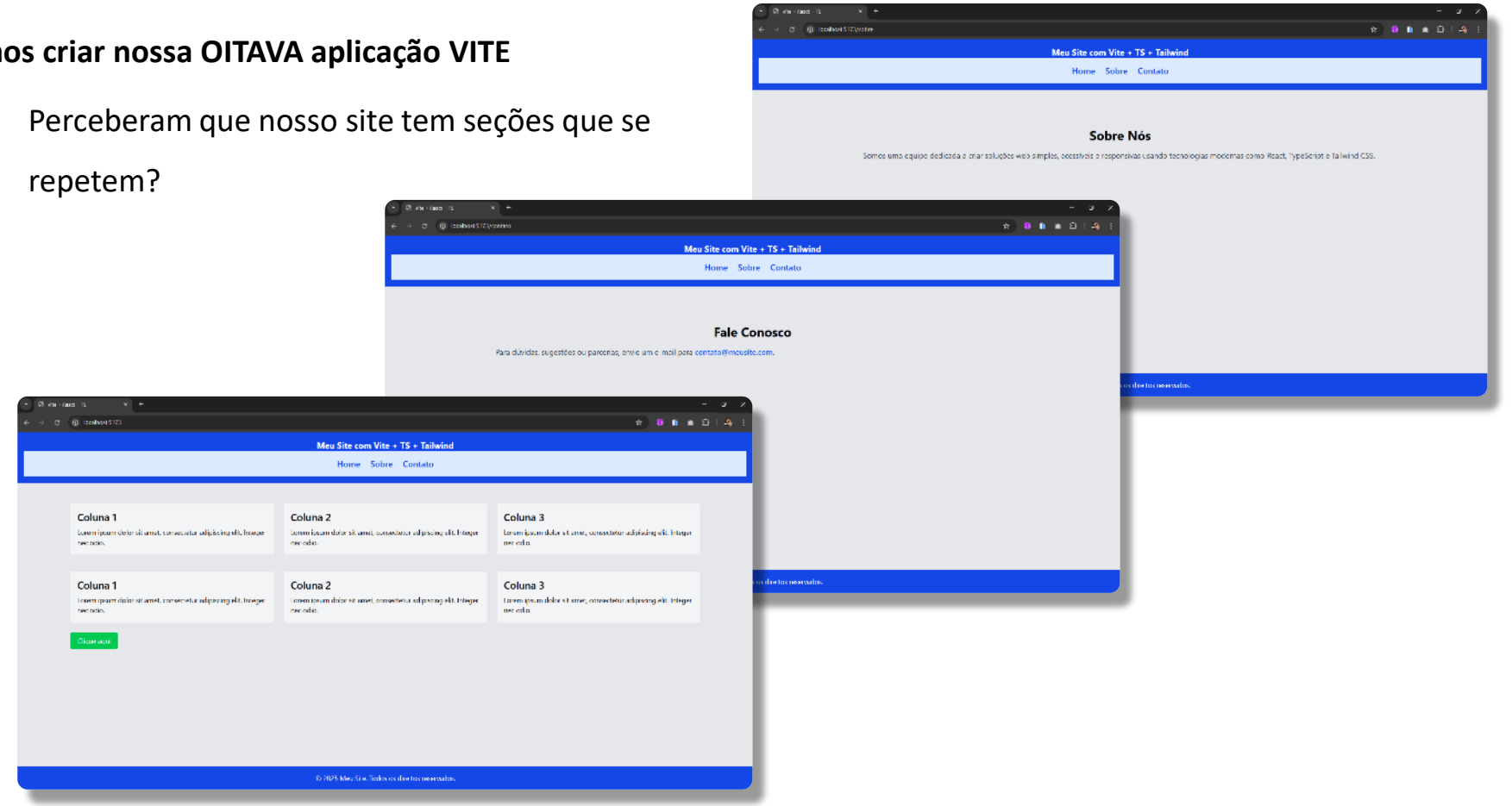
`'npm install react-router-dom'`



COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**

  - ✓ **Vamos criar nossa OITAVA aplicação VITE**
    - ✓ Perceberam que nosso site tem seções que se repetem?



COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

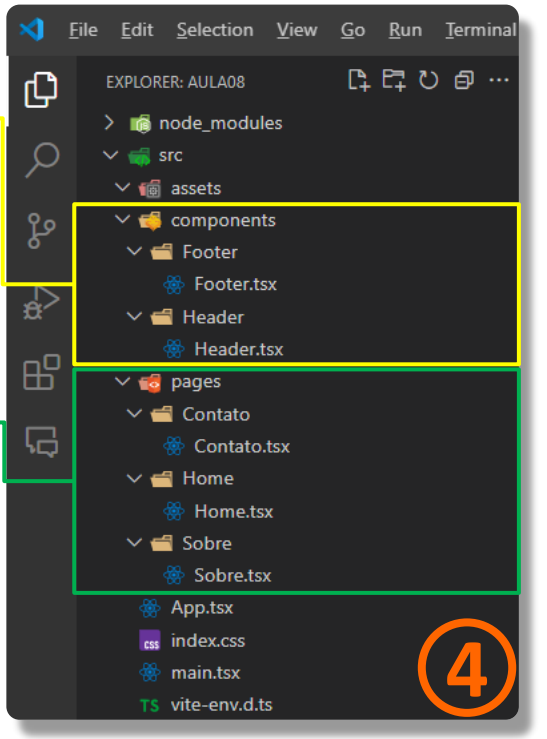
✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

4. Para que não repitamos códigos desnecessários em nossos arquivos, vamos dividir nossas páginas em componentes como na imagem.

   a) A **pasta components** serve para armazenar peças reutilizáveis da interface — cabeçalho, rodapé, botões, cards etc — que podem ser usados em várias páginas diferentes.

   b) A **pasta pages** guarda os conteúdos principais de cada rota do seu site — ou seja, cada página visível pelo usuário: Home, Sobre, Contato etc.

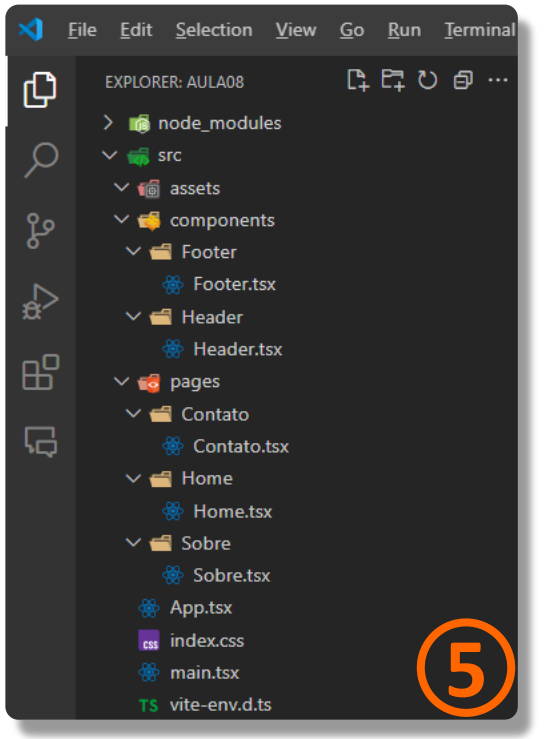


COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

5. Vamos utilizar o template disponibilizado, pois, já foi criado com Tailwind.



COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **Vamos criar nossa OITAVA aplicação VITE**

    6. Vamos criar o componente principal, ou seja, vamos atualizar o arquivo **APP.TSX**

6

```
const App = () => {
 return (
  <>
   <div className="flex flex-col min-h-screen h-screen w-screen">
    <header className="bg-blue-700 text-white p-4 text-center text-xl font-bold">
     <h1>Meu Site com Vite + TS + Tailwind</h1>
     <nav className="bg-blue-100 p-4 flex gap-6 justify-center text-blue-700 font-medium">
      <a href="/" className="hover:underline">Home</a>
      <a href="/sobre" className="hover:underline">Sobre</a>
      <a href="/contato" className="hover:underline">Contato</a>
     </nav>
    </header>
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

6. Vamos criar o componente principal, ou seja, vamos atualizar o arquivo **APP.TSX**

⑥

```
13.     <main className="flex-1 container mx-auto p-6">
14.      <div className="grid grid-cols-3 gap-6">
15.       <div className="bg-gray-100 p-4 rounded">
16.        <h2 className="text-2xl font-semibold mb-2">Coluna 1</h2>
17.        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
18.       </div>
19.       <div className="bg-gray-100 p-4 rounded">
20.        <h2 className="text-2xl font-semibold mb-2">Coluna 2</h2>
21.        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
22.       </div>
23.       <div className="bg-gray-100 p-4 rounded">
24.        <h2 className="text-2xl font-semibold mb-2">Coluna 3</h2>
25.        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

6. Vamos criar o componente principal, ou seja, vamos atualizar o arquivo **APP.TSX**

⑥

```
26.        </div>
27.       </div>
28.       <div className="grid grid-cols-3 gap-6 mt-10">
29.        <div className="bg-gray-100 p-4 rounded">
30.         <h2 className="text-2xl font-semibold mb-2">Coluna 1</h2>
31.         <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
32.        </div>
33.        <div className="bg-gray-100 p-4 rounded">
34.         <h2 className="text-2xl font-semibold mb-2">Coluna 2</h2>
35.         <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
36.        </div>
36.        <div className="bg-gray-100 p-4 rounded">
38.         <h2 className="text-2xl font-semibold mb-2">Coluna 3</h2>
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

6. Vamos criar o componente principal, ou seja, vamos atualizar o arquivo **APP.TSX**

6

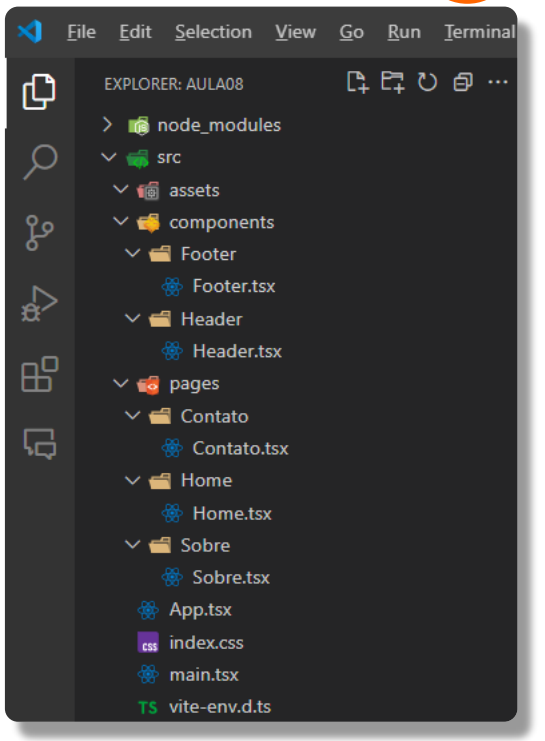
```
39.          <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
40.         </div>
41.        </div>
42.        <a href="#" className="bg-green-500 text-white px-4 py-2 rounded mt-6 inline-block">Clique
           aqui</a>
43.       </main>
44.       <footer className="bg-blue-700 text-white text-center p-4 mt-auto">
45.        &copy; {new Date().getFullYear()} Meu Site. Todos os direitos reservados.
46.       </footer>
47.      </div>
48.     </>
49.    );
50.   };
51.   export default App;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **Vamos criar nossa OITAVA aplicação VITE**

    7. Agora, vamos separar os códigos e criar os componentes e páginas de nossa aplicação.

7



COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **Vamos criar nossa OITAVA aplicação VITE**

8. Vamos criar o componente **componentes/Header/Header.tsx**

⑧

```
import { Link } from 'react-router-dom';
const Header = () => {
 return (
  <header className="bg-blue-700 text-white p-4 text-center text-xl font-bold">
   Meu Site com Vite + TS + Tailwind
   <nav className="bg-blue-100 p-4 flex gap-6 justify-center text-blue-700 font-medium">
    <Link to="/" className="hover:underline">Home</Link>
    <Link to="/sobre" className="hover:underline">Sobre</Link>
    <Link to="/contato" className="hover:underline">Contato</Link>
   </nav>
  </header>
 );
};
export default Header;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

9. Vamos criar o componente **componentes/Footer/Footer.tsx**

⑨

```
const Footer = () => {
 return (
  <footer className="bg-blue-700 text-white text-center p-4 mt-auto">
   &copy; {new Date().getFullYear()} Meu Site. Todos os direitos reservados.
  </footer>
 );
};
export default Footer;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

10. Vamos criar o componente **componentes/Card/Card.tsx**

```
import { ReactNode } from "react";
type CardProps = {
 children: ReactNode;
};
const Card = ({ children }: CardProps) => {
 return (
  <div className="bg-gray-100 p-4 rounded shadow-md">
   {children}
  </div>
 );
};
export default Card;
```

10

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**

  - ✓ **Explicando a propriedade CHILDREN**

    - ✓ **CHILDREN** é uma prop especial (propriedade reservada) que representa qualquer conteúdo que for colocado entre as tags de abertura e fechamento de um componente.

    - ✓ Imagine que você criou uma caixa personalizada (um componente). O **CHILDREN** é tudo o que você colocar dentro da caixa na hora de usá-la.

    - ✓ O **CHILDREN** não é uma propriedade do REACT-ROUTER-DOM, mas sim uma propriedade do REACT.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**

  - ✓ **Explicando a propriedade CHILDREN**

Por quê usar CHILDREN?

Componente	Função
Reutilização	Você pode mudar o conteúdo sem alterar o componente
Flexibilidade	Pode passar qualquer JSX: textos, imagens, listas, outros componentes...
Composição de layout	Ideal para criar *layouts*, *cards*, *caixas*, *modais*, *painéis*, etc.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **Vamos criar nossa OITAVA aplicação VITE**
    11. Vamos criar a página **pages/Contato/Contato.tsx**

11

```
1.  const Contato = () => {
2.   return (
3.    <section className="px-8 py-16 max-w-7xl mx-auto">
4.     <h1 className="text-3xl font-bold mb-4 text-center">Fale Conosco</h1>
5.     <p className="text-gray-700 text-lg text-justify">
6.      Para dúvidas, sugestões ou parcerias, envie um e-mail para <span className="text-blue-600 font-
        medium">contato@meusite.com</span>.
7.     </p>
8.    </section>
9.   );
10. };
11. export default Contato;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **Vamos criar nossa OITAVA aplicação VITE**
    12. Vamos criar a página **pages/Sobre/Sobre.tsx**

12

```
1.  const Sobre = () => {
2.   return (
3.    <section className="px-8 py-16 max-w-7xl mx-auto">
4.     <h1 className="text-3xl font-bold mb-4 text-center">Sobre Nós</h1>
5.     <p className="text-gray-700 text-lg text-justify">
6.      Somos uma equipe dedicada a criar soluções web simples, acessíveis e responsivas usando
        tecnologias modernas como React, TypeScript e Tailwind CSS.
7.     </p>
8.    </section>
9.   );
10. };
11. export default Sobre;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

13

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

13. Vamos criar a página **pages/Home/Home.tsx**

```
import Card from "../../components/Card/Card";
const Home = () => {
 return (
  <>
   <main className="flex-1 container mx-auto p-6">
    <div className="grid grid-cols-3 gap-6">
     <Card>
      <h2 className="text-2xl font-semibold mb-2">Coluna 1</h2>
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
     </Card>
     <Card>
      <h2 className="text-2xl font-semibold mb-2">Coluna 2</h2>
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
     </Card>
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

13

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

13. Vamos criar a página **pages/Home/Home.tsx**

```
15.      <Card>
16.       <h2 className="text-2xl font-semibold mb-2">Coluna 3</h2>
17.       <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
18.      </Card>
19.     </div>
20.     {/* Segunda linha com margem superior */}
21.     <div className="grid grid-cols-3 gap-6 mt-10">
22.      <Card>
23.       <h2 className="text-2xl font-semibold mb-2">Coluna 1</h2>
24.       <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
25.      </Card>
26.      <Card>
27.       <h2 className="text-2xl font-semibold mb-2">Coluna 2</h2>
28.       <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
29.      </Card>
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

13. Vamos criar a página **pages/Home/Home.tsx**

13

```
30.        <Card>
31.         <h2 className="text-2xl font-semibold mb-2">Coluna 3</h2>
32.         <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
33.        </Card>
34.       </div>
35.       <a href="#" className="bg-green-500 text-white px-4 py-2 rounded mt-6 inline-block">Clique
          aqui</a>
36.      </main>
37.     </>
38.   );
39.  };
40.  export default Home;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

14. Vamos MODIFICAR nosso componente inicial **APP.tsx**

14

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Header from './components/Header/Header';
import Footer from './components/Footer/Footer';
import Home from './pages/Home/Home';
import Sobre from './pages/Sobre/Sobre';
import Contato from './pages/Contato/Contato';
const App = () => {
 return (
  <BrowserRouter>
   <div className="flex flex-col min-h-screen h-screen w-screen">
    <Header />
    <Routes>
     <Route path="/" element={<Home />} />
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

14. Vamos MODIFICAR nosso componente inicial **APP.tsx**

14

```
14.        <Route path="/sobre" element={<Sobre />} />
15.        <Route path="/contato" element={<Contato />} />
16.       </Routes>
17.       <Footer />
18.      </div>
19.     </BrowserRouter>
20.    );
21.   };
22.   export default App;
```

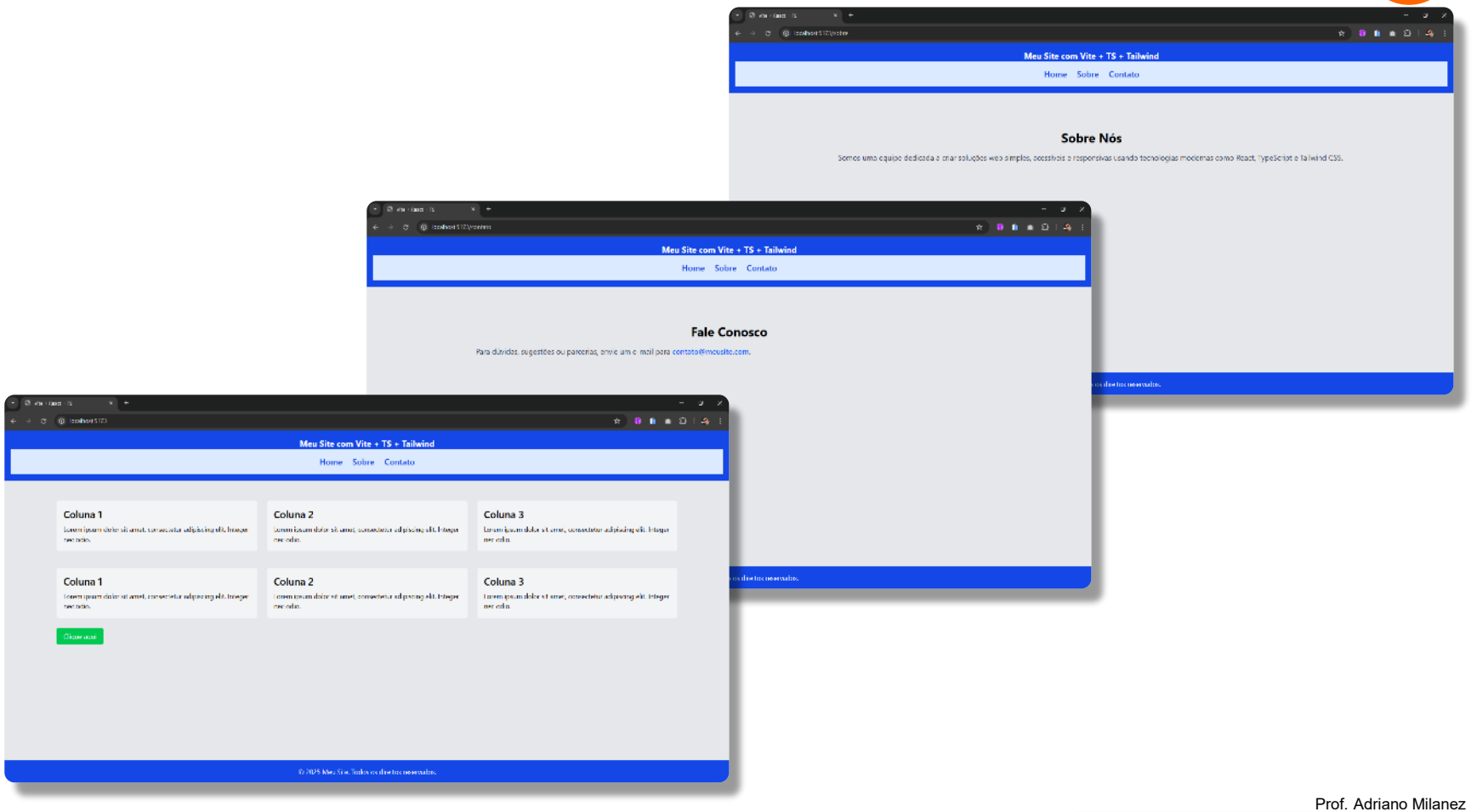
COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

15. Visualizando o resultado, devemos ter algo parecido com a imagem.

15



COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**

  - ✓ **<OUTLET />**
    - ✓ Vamos modificar nossa aplicação um pouco e trabalhar com **<OUTLET />**
      - ✓ O <Outlet /> é um componente especial do react-router-dom que marca o lugar onde os componentes filhos (rotas internas) serão renderizados.

  - ✓ **Quando usar <Outlet />:**
    - ✓ Quando tem um layout comum para várias páginas.
    - ✓ Quer que esse layout permaneça fixo enquanto o conteúdo da página muda conforme a rota.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **<OUTLET />**

Componente	Função
<Outlet />	Renderiza a **rota filha** dentro de uma rota pai com layout
Layout	Envolve o conteúdo comum (menu, header, footer)
Route com filhos	Define a estrutura de rotas aninhadas

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **<OUTLET />**

Critério	Layout no App.tsx	Layout com <Outlet />
🔧 **Implementação**	Mais simples e direto	Exige separação de componentes e rotas aninhadas
📁 **Organização do código**	Tudo misturado em um único lugar	Layout separado e reutilizável
♻️ **Reutilização de layout**	Limitada (um único layout)	Alta: suporta múltiplos layouts (ex: admin, público)
🔀 **Suporte a rotas aninhadas**	Difícil ou inviável	Totalmente suportado com <Outlet />
🧩 **Flexibilidade de estrutura**	Baixa (estrutura fixa)	Alta (páginas diferentes com layouts diferentes)
📈 **Escalabilidade do projeto**	Baixa (vira bagunça com muitas páginas)	Alta (ideal para apps grandes/modulares)
🔄 **Manutenção**	Mais difícil de ajustar o layout global	Mais fácil: altera o layout em um único lugar
👶 **Ideal para iniciantes?**	Sim, para projetos pequenos	Sim, mas exige leve curva de aprendizado
💼 **Ideal para produção?**	Não recomendado para apps com muitas rotas	Recomendado: padrão profissional

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **<OUTLET />**
    - ✓ **Conclusão**:
      - ✓ Use layout no App.tsx apenas se o projeto for muito simples e com poucas páginas.
      - ✓ Prefira usar <Outlet /> com um componente Layout para organizar, reutilizar e escalar melhor a estrutura da aplicação.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ Vamos criar nossa OITAVA aplicação VITE

16. Vamos criar o componente **componentes/Layout/Layout.tsx**

```
import { Outlet } from "react-router-dom";
import Header from "./Header";
import Footer from "./Footer";
const Layout = () => (
 <div className="flex flex-col min-h-screen h-screen w-screen">
  <Header />
  <main className="flex-1 container mx-auto p-6">
   <Outlet /> {/* Aqui entram as páginas */}
  </main>
  <Footer />
 </div>
);
export default Layout;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

17

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

17. Vamos modificar novamente o componente **APP.TSX**

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './pages/Home/Home';
import Sobre from './pages/Sobre/Sobre';
import Contato from './pages/Contato/Contato';
const App = () => {
 return (
  <BrowserRouter>
   <Routes>
    <Route path="/" element={<Layout />}>
     <Route index element={<Home />} />
     <Route path="sobre" element={<Sobre />} />
     <Route path="contato" element={<Contato />} />
    </Route>
   </Routes>
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **INTRODUÇÃO AO REACT-ROUTER-DOM**
  - ✓ **Vamos criar nossa OITAVA aplicação VITE**

17. Vamos modificar novamente o componente **APP.TSX**

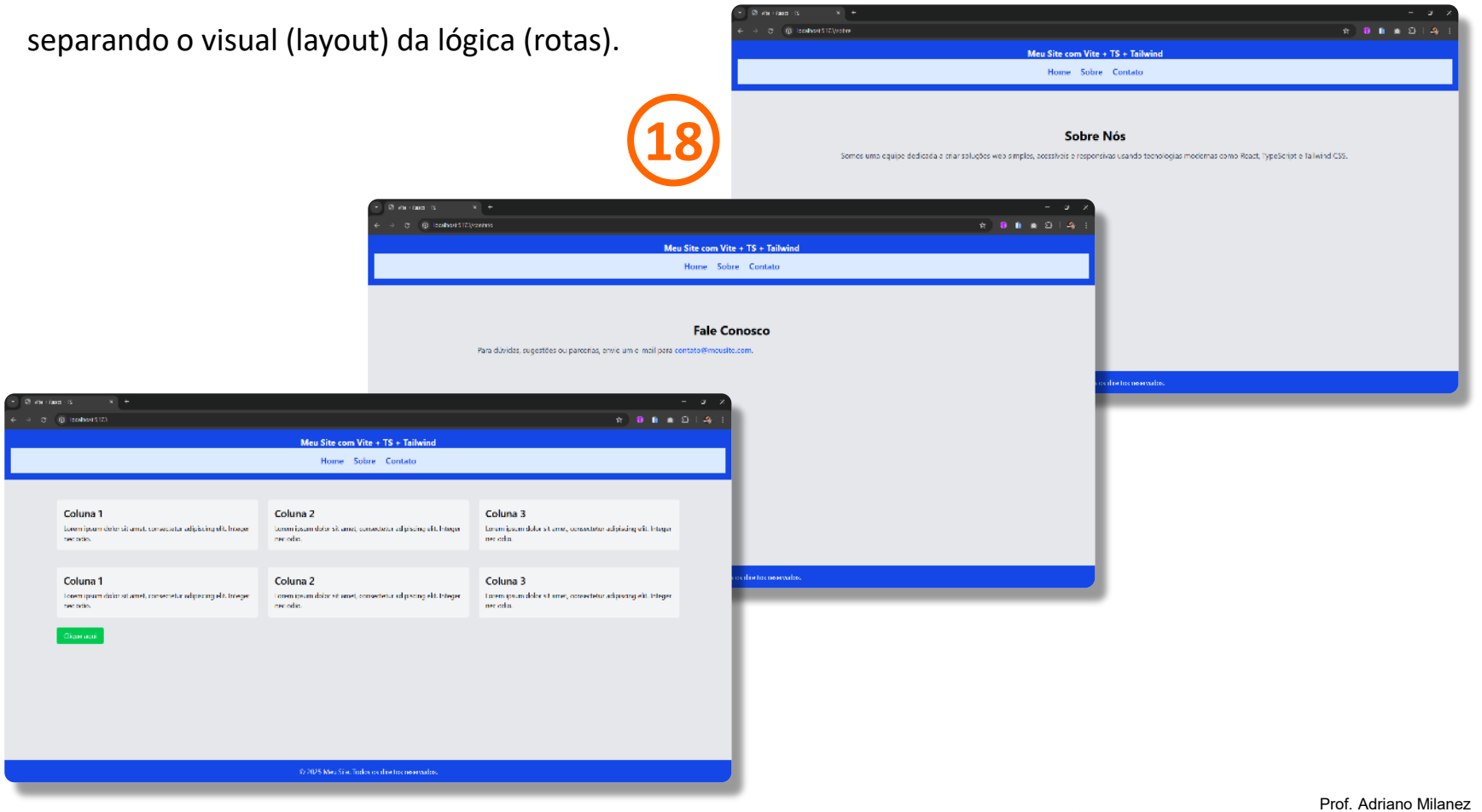
```
15.     </BrowserRouter>
16.    );
17.   };
18.   export default App;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

✓ INTRODUÇÃO AO REACT-ROUTER-DOM

✓ **Vamos criar nossa OITAVA aplicação VITE**

18. Visualizando o resultado final é o mesmo que o anterior, porém, nosso código fica mais organizado, separando o visual (layout) da lógica (rotas).



2.1

VITE – ROTAS ESTÁTICAS – EXERCÍCIOS

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- **Exercício 01 - Simples**

  - **Objetivo**: Entender como usar <Outlet /> para renderizar páginas com layout comum.

    - Crie um layout básico com cabeçalho, menu e rodapé utilizando Tailwind CSS.
    - Em seguida, utilize o componente <Outlet /> para renderizar uma página chamada Home dentro do layout.
    - Organize a estrutura em pastas components/ e pages/.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- **Exercício 02 - Intermediário**

  - **Objetivo**: Praticar rotas aninhadas com layout compartilhado.

    - Crie um layout básico com cabeçalho, menu e rodapé utilizando Tailwind CSS.
    - Em seguida, utilize o componente <Outlet /> para renderizar uma página chamada Home dentro do layout.
    - Organize a estrutura em pastas components/ e pages/.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- **Exercício 03 - Avançado**

  - **Objetivo**: Compreender como criar múltiplos layouts com <Outlet /> e organizar rotas complexas.

    - Implemente dois layouts distintos usando <Outlet />:
      - PublicLayout.tsx com páginas Home, Sobre, Contato
      - AdminLayout.tsx com páginas Dashboard, Usuários
    - Configure as rotas para que cada grupo de páginas use seu respectivo layout. Utilize Tailwind para dar identidade visual distinta aos dois layouts (ex: cores diferentes)

3

VITE – ROTAS DINÂMICAS

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**

  - ✓ **Rotas dinâmicas** são caminhos que mudam de acordo com valores passados na URL. Esses valores são parâmetros, e permitem renderizar o mesmo componente com dados diferentes.

  - ✓ **Exemplo**:
    - ✓ `/produto/1`
    - ✓ `/produto/2`
    - ✓ `/produto/banana`

  - ✓ **Por que usar rotas dinâmicas?**
    - ✓ Permite reutilizar componentes para exibir diferentes dados.
    - ✓ Ideal para páginas de detalhes: produtos, usuários, postagens etc.
    - ✓ Melhora a organização e escalabilidade do código

  - ✓ **Todas essas URLs podem usar a mesma página base, mas com conteúdo diferente, dependendo do valor**.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- **Por que usar ROTAS DINÂMICAS?**
  - **Sintaxe**
  - No roteamento criado em `App.tsx`, vamos identificar uma rota dinâmica com usando **:** para indicar o parâmetro.
    - `/produto/:id`

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- **O que são ROTAS DINÂMICAS?**
  - São caminhos que mudam de acordo com valores passados na URL.
  - Esses valores são parâmetros, e permitem renderizar o mesmo componente com dados diferentes.
  - Exemplo:
    - `/produto/1`
    - `/produto/2`
    - `/produto/banana`
  - **Todas essas URLs podem usar a mesma página base, mas com conteúdo diferente, dependendo do valor**.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

1. Vamos criar uma aplicação bem simples, sem formatação alguma, para que possamos entender o funcionamento das rotas dinâmicas e depois vamos modificar a aplicação que criamos para **rotas estáticas**. Vamos modificar o arquivo **`App.tsx`**

①

```
import { Routes, Route, Link, BrowserRouter } from 'react-router-dom';
import ProdutoDetalhe from './pages/ProdutoDetalhe/ProdutoDetalhe';
export default function App() {
 return (
  <>
   <BrowserRouter>
    <div>
     <h1>Produtos</h1>
     <nav>
      <Link to="/produto/1">Produto 1</Link> |  
      <Link to="/produto/2">Produto 2</Link>
     </nav>
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**
    1. Vamos criar uma aplicação bem simples, sem formatação alguma, para que possamos entender o funcionamento das rotas dinâmicas e depois vamos modificar a aplicação que criamos para **rotas estáticas**. Vamos modificar o arquivo **`App.tsx`**

①

```
13.        <Routes>
14.         <Route path="/produto/:id" element={<ProdutoDetalhe />} />
15.        </Routes>
16.       </div>
17.      </BrowserRouter>
18.     </>
19.    )
20.   }
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**
    1. Vamos criar uma aplicação bem simples, sem formatação alguma, para que possamos entender o funcionamento das rotas dinâmicas e depois vamos modificar a aplicação que criamos para **rotas estáticas**. Vamos modificar o arquivo **`App.tsx`**

①

```
13.        <Routes>
14.         <Route path="/produto/:id" element={<ProdutoDetalhe />} />
15.        </Routes>
16.       </div>
17.      </BrowserRouter>
18.     </>
19.    )
20.   }
```

O path define a URL que o usuário acessa, e o element é o componente React que será exibido nessa URL.

Percebam que o nome do caminho e do componente não precisam ser iguais, pois a URL é só um endereço, e o componente é o que aparece na tela.

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

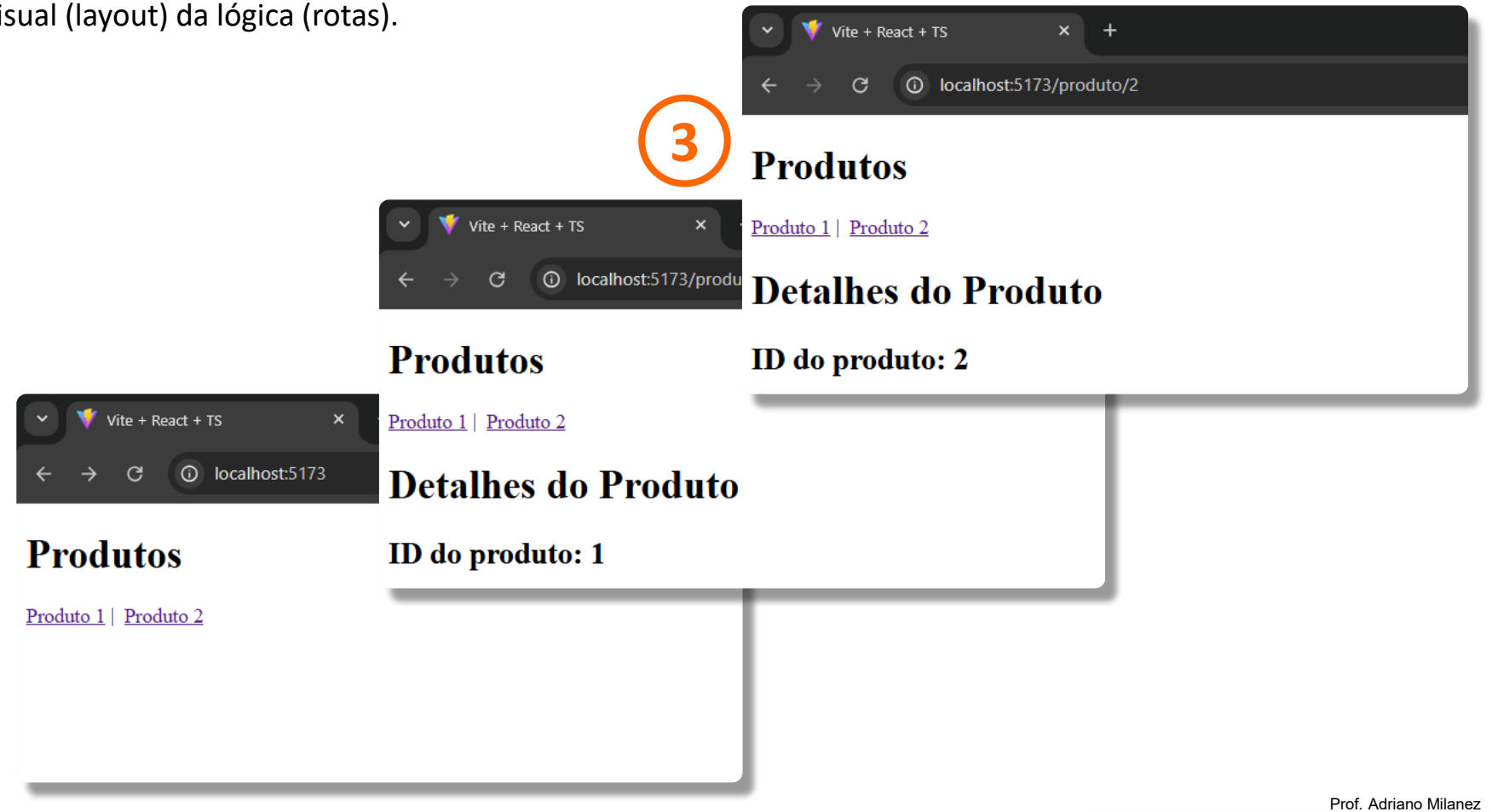
- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**
    2. Vamos criar o componente **`pages/ProdutoDetalhe/ProdutoDetalhe.tsx`**

②

```
1.  import { useParams } from "react-router-dom";
2.  const ProdutoDetalhe = () => {
3.   const { id } = useParams();
4.   return (
5.    <>
6.     <h1>Detalhes do Produto</h1>
7.     <h2>ID do produto: {id}</h2>
8.    </>
9.   );
10. };
11. export default ProdutoDetalhe;
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**
    3. Visualizando o resultado final é o mesmo que o anterior, porém, nosso código fica mais organizado, separando o visual (layout) da lógica (rotas).



COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

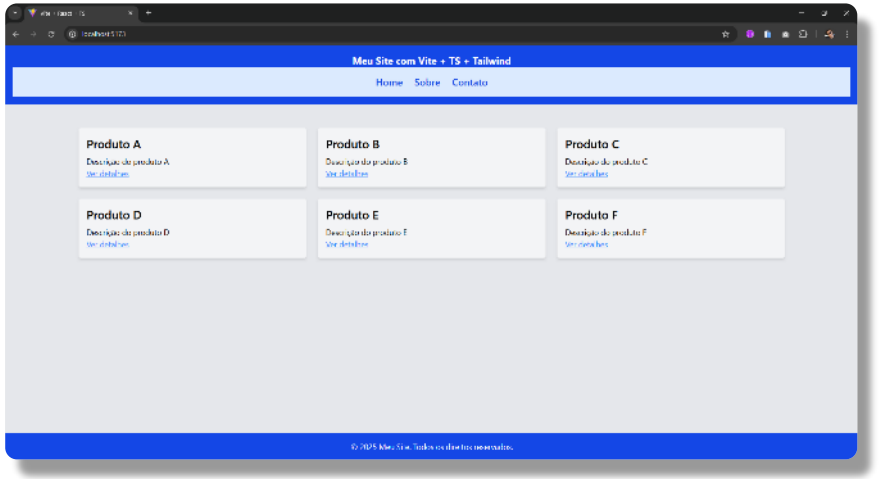
- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

    4. Vamos modificar a **aplicação** criada anteriormente **para rotas estáticas**, inserir rotas dinâmicas em nosso código e os fazer com que os CARDS seja carregados a partir de uma lista de produtos já fornecida.

④

Tela inicial

Tela final



COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

    5. Vamos modificar o arquivo **`App.tsx`**

```
1.  import { BrowserRouter, Routes, Route } from 'react-router-dom';
2.  import Home from './pages/Home/Home';
3.  import Sobre from './pages/Sobre/Sobre';
4.  import Contato from './pages/Contato/Contato';
5.  import ProdutoDetalhe from './pages/ProdutoDetalhe/ProdutoDetalhe';
6.  const App = () => {
7.   return (
8.    <BrowserRouter>
9.     <Routes>
10.      <Route path="/" element={<Layout />}>
11.       <Route index element={<Home />} />
12.       <Route path="sobre" element={<Sobre />} />
13.       <Route path="contato" element={<Contato />} />
14.       <Route path="ProdutoDetalhe/:id" element={<ProdutoDetalhe />} />
```

5

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

    5. Vamos modificar o arquivo **`App.tsx`**

```
15.        </Route>
16.       </Routes>
17.      </BrowserRouter>
18.    );
19.   };
20.   export default App;
```

5

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

6. Vamos modificar o arquivo **`Home.tsx`**

```
import { Link } from "react-router-dom";
import Card from "../../components/Card/Card";
const Home = () => {
 const produtos = [
  { id: 1, nome: "Produto A", descricao: "Descrição do produto A" },
  { id: 2, nome: "Produto B", descricao: "Descrição do produto B" },
  { id: 3, nome: "Produto C", descricao: "Descrição do produto C" },
  { id: 4, nome: "Produto D", descricao: "Descrição do produto D" },
  { id: 5, nome: "Produto E", descricao: "Descrição do produto E" },
  { id: 6, nome: "Produto F", descricao: "Descrição do produto F" },
 ];
 return (
  <>
```

6

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

    6. Vamos modificar o arquivo **`Home.tsx`**

```
14.     <main className="flex-1 container mx-auto p-6">
15.      {/* Primeira linha dinâmica */}
         <div className="grid grid-cols-3 gap-6">
16.       { produtos.map((produto) => (
17.        <Card key={produto.id}>
18.         <h2 className="text-2xl font-semibold mb-2">{produto.nome}</h2>
19.         <p>{produto.descricao}</p>
20.         <p>
21.          <Link to={`/ProdutoDetalhe/${produto.id}`} className="text-blue-500
             underline">Ver detalhes</Link>
22.         </p>
23.        </Card>
24.       ))}
```

6

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

    6. Vamos modificar o arquivo **`Home.tsx`**

```
25.        </div>
26.       </main>
27.      </>
28.    );
29.   };
30.   export default Home;
```

6

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

7. Vamos criar o arquivo **`pages/ProdutoDetalhe/ProdutoDetalhe.tsx`**

```
import { useParams } from "react-router-dom";
const ProdutoDetalhe = () => {
 const produtos = [
  { id: 1, nome: "Produto A", descricao: "Descrição do produto A" },
  { id: 2, nome: "Produto B", descricao: "Descrição do produto B" },
  { id: 3, nome: "Produto C", descricao: "Descrição do produto C" },
  { id: 4, nome: "Produto D", descricao: "Descrição do produto D" },
  { id: 5, nome: "Produto E", descricao: "Descrição do produto E" },
  { id: 6, nome: "Produto F", descricao: "Descrição do produto F" },
 ];
 const { id } = useParams();
```

7

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

7. Vamos criar o arquivo **`pages/ProdutoDetalhe/ProdutoDetalhe.tsx`**

```
const produto = produtos.find((p) => p.id === Number(id));
if (!produto) {
 return (
  <>
   <section className="px-8 py-16 max-w-7xl mx-auto text-center">
    <h1 className="text-3xl font-bold mb-4">Produto não encontrado! 😢</h1>
   </section>
  <>
 );
}
return (
 <>
  <section className="px-8 py-16 max-w-7xl mx-auto">
   <h1 className="text-3xl font-bold mb-6 text-center">Detalhes do Produto</h1>
```

⑦

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

7. Vamos criar o arquivo **`pages/ProdutoDetalhe/ProdutoDetalhe.tsx`**

```
25.       <div className="bg-white shadow-md rounded-lg p-6">
26.        <h2 className="text-2xl font-bold mb-2">{produto.nome}</h2>
27.        <p className="text-gray-700 text-lg mb-4">{produto.descricao}</p>
28.        <p className="text-sm text-gray-500">ID do produto: {produto.id}</p>
29.       </div>
30.      </section>
31.     </>
32.   );
33.  };
34.  export default ProdutoDetalhe;
```

7

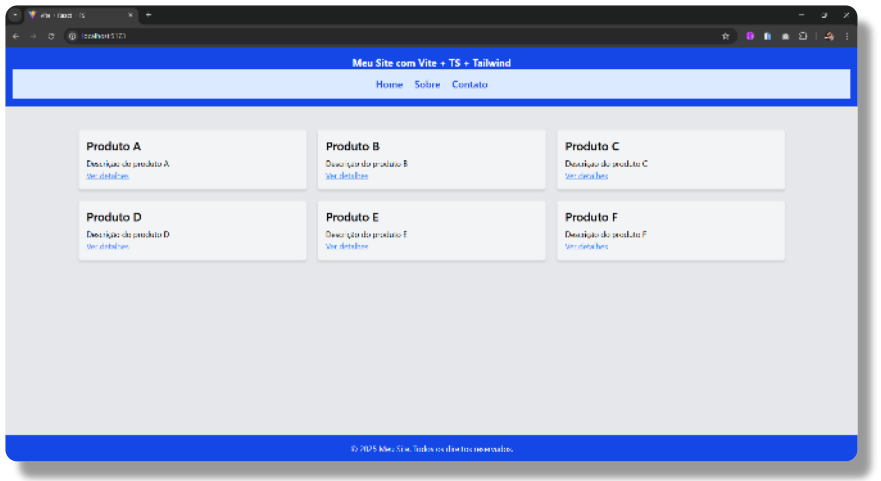
COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

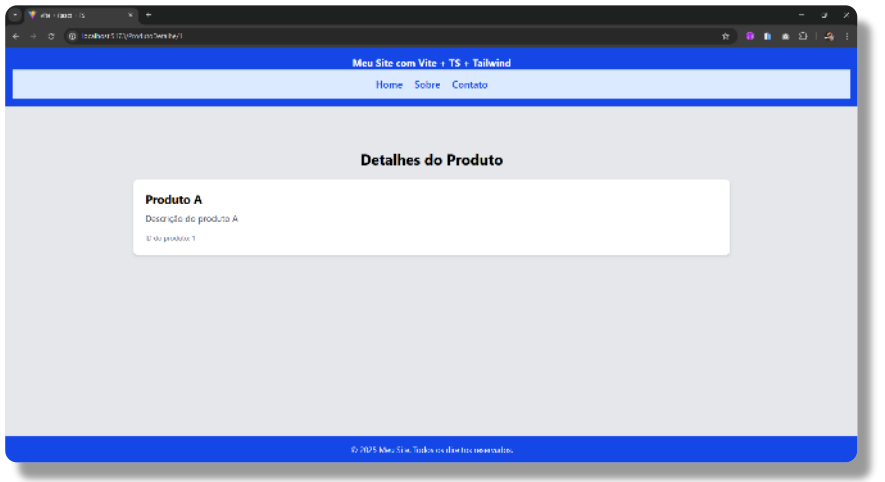
    8. Como resultado temos:

8

Tela inicial



Tela final



COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

9. Perceberam que em nossos componentes **`Home.tsx`** e **`ProdutoDetalhe.tsx`**, existem códigos repetidos?

   Vamos retirar esse trecho dos DOIS componentes.

9

```
const produtos = [
  { id: 1, nome: "Produto A", descricao: "Descrição do produto A" },
  { id: 2, nome: "Produto B", descricao: "Descrição do produto B" },
  { id: 3, nome: "Produto C", descricao: "Descrição do produto C" },
  { id: 4, nome: "Produto D", descricao: "Descrição do produto D" },
  { id: 5, nome: "Produto E", descricao: "Descrição do produto E" },
  { id: 6, nome: "Produto F", descricao: "Descrição do produto F" },
];
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

    10. Vamos melhor nosso código, recortando esse trecho e criando um arquivo JSON, **`src/data/produtos.json`**

10

```
1.  [
2.    { "id": 1, "nome": "Produto A", "descricao": "Descrição do produto A" },
3.    { "id": 2, "nome": "Produto B", "descricao": "Descrição do produto B" },
4.    { "id": 3, "nome": "Produto C", "descricao": "Descrição do produto C" },
5.    { "id": 4, "nome": "Produto D", "descricao": "Descrição do produto D" },
6.    { "id": 5, "nome": "Produto E", "descricao": "Descrição do produto E" },
7.    { "id": 6, "nome": "Produto F", "descricao": "Descrição do produto F" }
8.  ]
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

11. Vamos modificar o arquivo **`Home.tsx`**

```
import { Link } from "react-router-dom";
import Card from "../../components/Card/Card";
import produtos from "../../data/produtos.json";
const Home = () => {
 return (
  <>
   <main className="flex-1 container mx-auto p-6">
    {/* Primeira linha dinâmica */}
    <div className="grid grid-cols-3 gap-6">
     { produtos.map((produto) => (
      <Card key={produto.id}>
       <h2 className="text-2xl font-semibold mb-2">{produto.nome}</h2>
       <p>{produto.descricao}</p>
       <p>
```

11

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

    11. Vamos modificar o arquivo **`Home.tsx`**

```
14.            <Link to={`/ProdutoDetalhe/${produto.id}`} className="text-blue-500
               underline">Ver detalhes</Link>
15.          </p>
16.         </Card>
17.        ))}
18.       </div>
19.      </main>
20.     </>
21.    );
22.   };
23.   export default Home;
```

11

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

12. Vamos fazer o mesmo no arquivo **`pages/ProdutoDetalhe/ProdutoDetalhe.tsx`**

12

```
import { useParams } from "react-router-dom";
import produtos from "../../data/produtos.json";
 const ProdutoDetalhe = () => {
  const { id } = useParams();
 const produto = Produtos.find((p) => p.id === Number(id));
 if (!produto) {
  return (
   <>
    <section className="px-8 py-16 max-w-7xl mx-auto text-center">
     <h1 className="text-3xl font-bold mb-4">Produto não encontrado! 😢</h1>
    </section>
   <>
  );
 }
```

COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

12. Vamos criar o arquivo **`pages/ProdutoDetalhe/ProdutoDetalhe.tsx`**

12

```
12.   return (
13.     <>
14.       <section className="px-8 py-16 max-w-7xl mx-auto">
15.        <h1 className="text-3xl font-bold mb-6 text-center">Detalhes do Produto</h1>
16.        <div className="bg-white shadow-md rounded-lg p-6">
            <h2 className="text-2xl font-bold mb-2">{produto.nome}</h2>
17.         <p className="text-gray-700 text-lg mb-4">{produto.descricao}</p>
18.         <p className="text-sm text-gray-500">ID do produto: {produto.id}</p>
19.        </div>
20.      </section>
21.     </>
22.   );
23.  };
24.  export default ProdutoDetalhe;
```

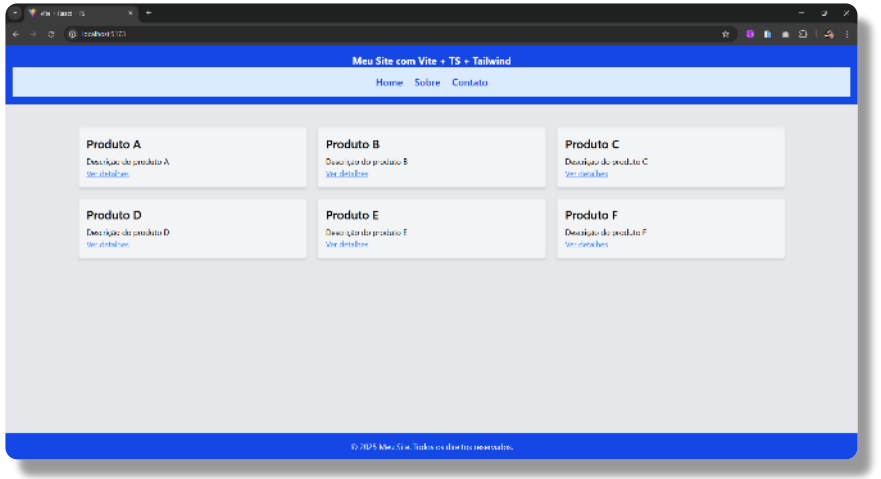
COMO CRIAR NAVEGAÇÃO ENTRE COMPONENTES

- ✓ **O que são ROTAS DINÂMICAS?**
  - ✓ **Vamos criar mais uma aplicação VITE**

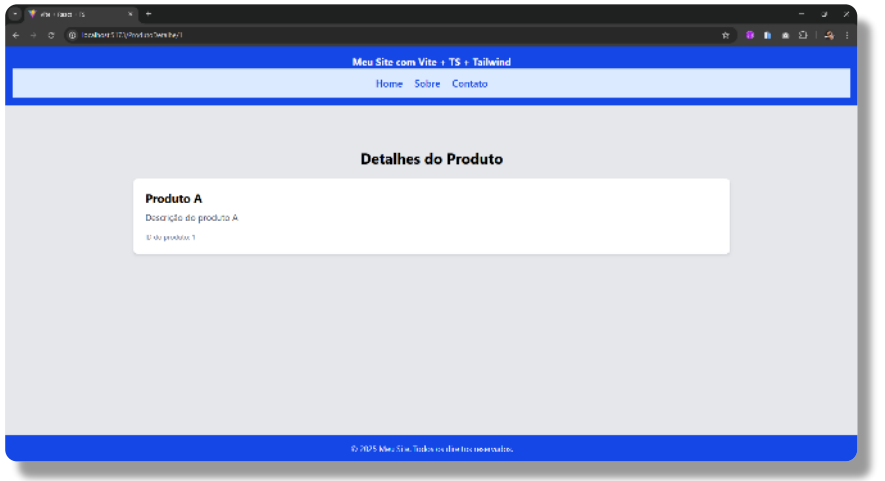
    13. O resultado final será o mesmo, porém, os **dados** dos produtos ficam **organizados separadamente**, facilitando **manutenção**, **escalabilidade** e até futura integração com **API**.

13

Tela inicial



Tela final



Referências Bibliográficas

ABREU, Luis. Typescript - O Javascript moderno para criação de aplicações. Editora FCA, 2017.

ADRIANO, T. S. Guia prático de TypeScript. São Paulo: Casa do Código, 2021.

ANTONIO, C. Pro React: Build Complex Front-End Applications in a Composable Way With React. Apress, 2015.

BANKS, Adam. Learning React: Functional Web Development with React and Redux. 2. ed. Sebastopol: O'Reilly Media, 2020.

BOSWELL, D; FOUCHER, T. The Art of Readable Code: Simple and Practical Techniques for Writing Better Code. Estados Unidos: O'Reilly Media, 2012.

BUNA, S. React Succinctly. Estados Unidos: [s.n], 2016. Disponível em: <www.syncfusion.com/ebooks/reactjs_succinctly>. Acesso em: 12 de janeiro de 2025.

CEDDIA, Dave. Pure React: A step-by-step guide to mastering React. 1. ed. [S.l.]: Self-published, 2018.

CHEHADE, Adib; CHEHADE, Nadia. Fullstack React with TypeScript and Next.js: Build scalable and production-ready full stack web apps. 1. ed. Birmingham: Packt Publishing, 2023.

FACEBOOK. React: Getting Started. React Docs, 2019a. Disponível em: <reactjs.org/docs/react-api.html>. Acesso em: 13 de julho de 2025.

_____. React Without ES6. React Docs, 2019b. Disponível em: <reactjs.org/docs/react-without-es6.html>. Acesso em: 10 de julho de 2025.

FERNANDES, Flavio. Desenvolvimento Front-end com React e TypeScript. 1. ed. São Paulo: Novatec Editora, 2021.

_____. React Without JSX. React Docs, 2019c. Disponível em: <reactjs.org/docs/react-without-jsx.html>. Acesso em: 10 de julho de 2025.

FREEMAN, Eric; ROBSON, Elisabeth. Use a Cabeça! Programação em HTML5. Rio de Janeiro: Editora Alta Books, 2014.

FREEMAN, Eric; ROBSON, Elisabeth. Head First JavaScript Programming. 1. ed. Sebastopol: O'Reilly Media, 2014.

GACKENHEIMER, C. Introduction to React: Using React to Build scalable and efficient user interfaces.[s.i.]: Apress, 2015.

GOLDBERG, Josh. Aprendendo TypeScript: Melhore Suas Habilidades de Desenvolvimento web Usando JavaScript Type-Safe. São Paulo: Novatec. 2022.

HARTIGAN, Nathan. Vite Handbook: A beginner's guide to fast modern web development. 1. ed. [S.l.]: Leanpub, 2022.

HUDSON, P. Hacking with React. 2016. Disponível em: <www.hackingwithreact.com/read/1/3/introduction-to-jsx>. Acesso em: 13 julho de 2025.

KOSTRZEWA, D. Is React.js the Best JavaScript Framework in 2018? 2018. Disponível em: <hackernoon.com/is-react-js-the-best-JavaScript-framework-in-2018-264a0eb373c8>. Acesso em: 25 de julho de 2025.

Referências Bibliográficas

MARDER, Carl. React and React Native: A complete hands-on guide to modern web and mobile development with React.js. 3. ed. Birmingham: Packt Publishing, 2020.

MDN WEB DOCS. Guia JavaScript. Disponível em <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide>. Acessado em 25 de julho de 2025.

MICROSOFT. TypeScript: JavaScript with syntax for types. Disponível em: <https://www.typescriptlang.org>. Acesso em: 22 de julho de 2025.

NELSON, J. Learn React's Fundamentals Without the Buzzwords? 2018. Disponível em: <jamesknelson.com/learn-react-fundamentals-sans-buzzwords>. Acesso em: 12 julho de 2025.

NIELSEN, J. Response Times: The 3 Important Limits. 1993. Disponível em: <www.nngroup.com/articles/response-times-3-important-limits>. Acesso em: 10 julho de 2025.

O'REILLY, T. What is Web 2.0: Design Patterns and Business Models for the Next Generation of Software. 2005. Disponível em: <www.oreilly.com/pub/a/web2/archive/ what-is-web-20.html#mememap>. Acesso em: 10 de julho de 2025.

PANDIT, N. What Is ReactJS and Why Should We Use It? 2018. Disponível em: <www.c-sharpcorner.com/article/what-and-why-reactjs>. Acesso em: 12 de julho de 2025.

PATEL, Vishal Layka. Pro TypeScript: Application-Scale JavaScript Development. 1. ed. Berkeley: Apress, 2014.

RAUSCHMAYER, A. Speaking JavaScript: An In-Depth Guide for Programmers. Estados Unidos: O'Reilly Media, 2014.

REMIX. React Router Documentation. Disponível em: <https://reactrouter.com.> Acesso em: 19 de julho de 2025.

RICHARDS, Mike. Tailwind CSS: From Zero to Production. 1. ed. [S.l.]: Independently published, 2021.

RICOY, L. Desmitificando React: Uma Reflexão para Iniciantes. 2018. Disponível em: <medium.com/trainingcenter/desmitificando-react-uma-reflex%C3%A3o-para-iniciantes-a57af90b6114>. Acesso em: 13 julho de 2025.

RIEDEL, Alex. TypeScript Quickly. 1. ed. Shelter Island: Manning Publications, 2020.

SHARMA, Nabendu Biswas; UDAYAN, Kuldeep. Building Microfrontends with React 18 and Next.js 13: A complete guide to building modern microfrontend-based web apps using React and Next.js. 1. ed. Birmingham: Packt Publishing, 2023.

STACKOVERFLOW. Most Popular Technologies: Web Frameworks. Developer Survey Results, StackOverflow, 2019. Disponível em: <insights.stackoverflow.com/survey/ 2019#technology>. Acesso em: 13 de julho de 2025.

_____. Sticky Footer with Tailwind CSS. Disponível em: https://stackoverflow.com/questions/43556907/sticky-footer-with-tailwind-css. Acesso em: 10 de julho de 2025.

SWC. Rust-based platform for the Web. 2024 Disponível em <https://swc.rs/>. Acessado em 23 de julho de 2025.

Referências Bibliográficas

TAILWIND CSS. Tailwind CSS Documentation. Disponível em: https://tailwindcss.com/docs. Acesso em: 10 de julho de 2025.

_____. Making a Full Height Layout. Disponível em: https://tailwindcss.com/docs/min-height#making-a-full-height-layout. Acesso em: 10 de fevereiro de 2025.

_____. Curso gratuito de Tailwind CSS. Disponível em: https://play.tailwindcss.com. Acesso em: 10 de julho de. 2025.

VITE. Documentação oficial do Vite. Disponível em: https://vitejs.dev. Acesso em: 22 de julho de 2025.

WIKIPEDIA. JavaScript. Disponível em <https://pt.wikipedia.org/wiki/JavaScript>. Acessado em 20 de julho de 2025.

WILSON, Michael. Modern Full-Stack Development: Using TypeScript, React, Node.js, Webpack, and Docker. 1. ed. Sebastopol: O’Reilly Media, 2020.

Dúvidas?

Críticas?

Sugestões?

Ameaças?